

API

one.Joint control

For a serial multi-joint robot, the joint control is to control the variables of each joint of the robot arm so as to make each joint reaches a target position at a certain speed.

1 Single joint control

1.1 Sending the angle of single joint

WriteAngle(Joint joint, double value, int speed = DefaultSpeed)[example](#)

Return value: no parameter

parameter description: Parameter 1: joint number (1-6) Parameter 2: angle (-170°- 170°)

Parameter 3: speed (0-100), the example with 30 as a default

example:

```
`mycobot::MyCobot::I().WriteAngle(mycobot::Joint::J1, 10, 30);
```

1.2 Obtain the minimum Angle of single joint

GetJointMin(Joint joint) [example](#)

Function: to read the minimum angle of a joint

Return value: double type, the minimum angle (the minimum angle that the joint can reach)

Parameter description: joint number (1-6)

example:

```
mycobot::MyCobot::I().GetJointMin(mycobot::Joint::J1);
```

1.3 Get the maximum Angle of single joint

GetJointMax(Joint joint)[example](#)

Function: to read the maximum angle of a joint

Return value: double type, the maximum angle (the maximum angle that the joint can reach)

Parameter description: joint number (1-6)

example:

```
mycobot::MyCobot::I().GetJointMax(mycobot::Joint::J1);
```

1.4 Perform a Jog on the specified joint

JogAngle(Joint joint, int direction, int speed = DefaultSpeed)[example](#)

Function: to make a joint move until Jogstop or reaching a limited position

Return value: none

Parameter description: Parameter 1: joint number (1-6), Parameter 2: direction (1 - positive direction, and 0 - negative direction), and Parameter 3: speed (Default 30, ranging from 0 to 100).

Notice: This API will make the robot arm move in the forward and reverse directions, and it will stop moving after reaching the limited position or when JogStop is called during the movement

example:

```
mycobot::MyCobot::I().JogAngle(mycobot::Joint::J1, 1, 30);
```

1.5 Jog motion with incremental position on specified joint

JogAngleIncrement(Joint joint, double increment, int speed = DefaultSpeed)[example](#)

Function: to make a joint move by a set angle increment

Return value: none

Parameter description: Parameter 1: joint number (1-6), Parameter 2: joint increment value, and Parameter 3: speed (with a default of 30, and ranging from 0 to 100). Notice: The robot arm performs discrete motion; for example, the current coordinate of the joint 1 is -100, and the incremental value is 50. After the movement, the coordinate of the joint 1 will be 50

example:

```
mycobot::MyCobot::I().JogAngleIncrement(mycobot::Joint::J1, 10, 30);
```

2 Multi-joint control

2.1 Get the angles of all joints

GetAngles()[example](#)

Return value: Angles type

parameter description: no example:

```
mycobot::Angles angles= mycobot::MyCobot::I().GetAngles();
```

2.2 Send the angles of all joints

WriteAngles(const Angles& angles, int speed = DefaultSpeed)[example](#)

Return value: no

parameter description: Parameter 1: all angles (std::array, ranging from -170° to 170°) Parameter 2: speed (0-100), the example with 30 as a default

```
mycobot::Angles goal_angles = { 5, 5, 5, 5, 5, 5 };
mycobot::MyCobot::I().WriteAngles(goal_angles, 30);
mycobot::Angles goal_angles = { 5, 5, 5, 5, 5, 5 };
mycobot::MyCobot::I().WriteAngles(goal_angles, 30);
```

two.Coordinate control

Coordinate control is to make the robot arm move to a specified point with a specified posture, which is divided into x, y, z, rx, ry, rz. X, Y and Z represents the position of the robot arm head in space (The coordinate system is Cartesian coordinate system); [rx,ry,rz] represents the posture of such head at this point (The coordinate system is Euler coordinates).

1 Single parameter control

1.1 Sending single parameter coordinates

WriteCoord(Axis axis, double value, int speed = DefaultSpeed) [example](#)

Return value: no

parameter description: Parameter 1: coordinate number (Axis enumeration type, int: 1-6 (X-RZ)); Parameter 2: coordinate (X, Y and Z range from -300 to 300.00 in mm; RX, RY, RZ ranges from -180 to 180); Parameter 3: speed (0-100), the example with 30 as a default

example:

```
mycobot::MyCobot::I().WriteCoord(mycobot::Axis::X, 10, 30);
```

1.2 Jog on the specified axis

JogCoord(Axis axis, int direction, int speed = DefaultSpeed) [example](#)

Function: to make the robot arm move in the direction of a coordinate axis

Return value: none

Parameter description: Parameter 1: coordinate number (1-6, xyz rx ry rz), Parameter 2: direction (1 - positive direction, and 0 - negative direction), and Parameter 3: speed (Default is 30, ranging from 0 to 100). Notice: This API will make the robot arm move in the forward and reverse directions of the coordinate axis, and it will stop moving after reaching a limited position or when JogStop is called during the movement

example:

```
mycobot::MyCobot::I().JogCoord(mycobot::Axis::X, 1, 30);
```

1.3 Jog with incremental position on the specified axis

JogCoordIncrement(Axis axis, double increment, int speed = DefaultSpeed) [example](#)

Function: to make a coordinate move by a set coordinate increment

Return value: none

Parameter description: Parameter 1: coordinate number (1-6, xyz rx ry rz), Parameter 2: coordinate increment value, and Parameter 3: speed (with a default of 30, and ranging from 0 to 100). Notice: The robot arm performs discrete motion; for example, the current coordinate of X-axis is 100, and the incremental value is 50. After the movement, the coordinate of X-axis will be 150

example:

```
mycobot::MyCobot::I().JogCoordIncrement(mycobot::Axis::X, 1, 30);
```

2 Multi-parameter control

2.1 Get all coordinates

GetCoords() [example](#)

Return value: Coords type parameter description: no example: mycobot::Coords coords = mycobot::MyCobot::I().GetCoords();

2.2 Send all coordinates

WriteCoords(const Coords& coords, int speed = DefaultSpeed) [example](#)

Return value: no

parameter description: Parameter 1: coordinates (X, Y and Z range from -300 to 300.00 in mm; RX, RY and RZ range from -180 to 180); Parameter 2: speed (0-100), the example with 30 as a default example:

```
mycobot::Coords goal_coords = {5, 5, 5, 5, 5, 5 };
mycobot::MyCobot::I().WriteCoords(goal_coords, 30);
mycobot::Coords goal_coords = {5, 5, 5, 5, 5, 5 };
mycobot::MyCobot::I().WriteCoords(goal_coords, 30);
```

tree.IO control

There are pins on the M5Stack-basic at the bottom of the robot arm and on the Atom at the end. The adsorption pump and other tools can be controlled by setting high and low levels of the pins through io control. For the numbers of input and output pins for each type of robot arm, refer to the following description table.

Table of Description of Input and Output Pins on the M5Stack-basic

Robot arm model	Input pin number	Output pin number
myCobot 280-M5	35、 36	2、 5、 26
myCobot 320-M5	35、 36	5、 15

Table of Description of Input and Output Pins on the Atom

Robot arm model	Input pin number	Output pin number
myCobot 280-M5	19、 22	23、 33
myCobot 320-M5	none	none

1 M5Stack-basic io control (m5)

1.1 Setting the high and low levels of output io

SetBasicOut(int pin_number, int pin_signal)[example](#)

Return value: no

parameter description: Parameter 1: pin number (basic output pin number); Parameter 2: state (0 - low level; 1- high level)

Case: set the output pin 2 to high level

example:

```
mycobot::MyCobot::I().SetBasicOut(2, 1);
```

1.2 Getting the state of input io

GetBasicIn(int pin_number)[example](#)

Return value: pin state (0-low level; 1-high level)

Parameter description: pin number (basic input pin number)

Case:

example:

```
mycobot::MyCobot::I().GetBasicIn(35);
```

2 Atom io Control

Note: 320m5 has no atom io, so the module API is not used.

2.1 Setting the high and low levels of output io

SetDigitalOut(int pin_number, int pin_signal) [example](#)

Return value: no

parameter description: Parameter 1: pin number (atom output pin number); Parameter 2: state (0 - low level; 1- high level)

Case:

example:

```
mycobot::MyCobot::I().SetDigitalOut(23, 1);
```

2.2 Getting the state of input io

GetDigitalIn(int pin_number) [example](#)

Return value: pin state (0-low level; 1-high level)

Parameter description: pin number (atom input pin number)

Case:

```
mycobot::MyCobot::I().GetDigitalIn(19);
```

four.Other API introductions

1 Instantiate MyCobot

I()

Function: Instantiate MyCobot

return value: MyCobot type, single instance of myCobot object

parameter description: none

Note: When calling the following API, it is not necessary to do separate instantiation, and just call this API

2 Overall running status of Robot

2.1 PowerOn()

Function: to power up the robot arm

Return value: none

parameter description: none

Note: After the robot arm is powered up, it cannot be moved manually

2.2 PowerOff()

Function: to cut off the power for the robot arm

Return value: none

parameter description: none

Note: After the robot arm is powered up, if you want to move it manually, you may call this API

2.3 SetFreeMoveMode(bool free_move = true)

Function: to set free movement mode

Return value: none

parameter description: enable or disable free movement; true – open free movement, and false – close free automatic

Note: After free movement is enabled, you may move the robot arm manually; and at the same time the light on the atom will turn yellow, and it will turn green when it is turned off

2.4 IsFreeMoveMode()

Function: to check whether the current free movement mode

Return value: bool type, true - free movement is enabled, false - free movement is not enabled

parameter description: none

2.5 IsControllerConnected()

Function: to check whether the system is normal

return value: none

parameter description: bool type, if false is returned, the robot arm cannot be controlled

3 MDI program control mode

3.1 IsInPosition(const Coords& coords, bool is_linear = true)

Function: to check whether the robot arm reaches a specified point (angle or coordinate)

Return value: bool type; if false is returned , it means failing to reach the specified point; and if true is returned, it means having reached the specified point

Parameter description: Parameter 1: all angles or coordinates; Parameter 2: 0 or 1 (Coordinate is 1 (true), and angle is 0 (false))

3.2 IsMoving()

Function: to detect whether the robot arm is moving

Return value: bool type, true - moving, false - not moving

Parameter description: none

3.3 StopRobot()

Function: to stop the movement of the robot arm. When the robot arm is moving, you may stop it by calling this API

Return value: none

Parameter description: none

4 Running Auxiliary Information

4.1 GetSpeed()

Function: to get the speed of the robot arm

Return value: int type, the movement speed of the robot arm (0-100)

Parameter description: none

4.2 SetSpeed(int percentage)

Function: to set the movement speed of the robot arm

Return value: none

Parameter description: the movement speed of the robot arm (0-100)

4.3 GetJointMin(Joint joint)

Function: to read the minimum angle of a joint

Return value: double type, the minimum angle (the minimum angle that the joint can reach)

Parameter description: joint number (1-6)

4.4 GetJointMax(Joint joint)

Function: to read the maximum angle of a joint

Return value: double type, the maximum angle (the maximum angle that the joint can reach)

Parameter description: joint number (1-6)

4.5 SleepSecond(unsigned time)

Function: to wait

Return value: none

Parameter description: time unit is second